

IT7075: Applied AI for Cybersecurity

Instructor: Chengcheng Li • University of Cincinnati
School of Information Technology

LangChain

Mini Project: Rebuild Your RAG with LangChain

Work through the LangChain pipeline, rebuild your RAG on the same data, and compare the two ways of building it.

Runs on your own machine or Colab • 100 points

1. What this project is about

In the RAG module you built a retrieval system by hand: your own chunking loop, your own calls to the vector database, your own prompt assembly, your own scoring. LangChain does all of that for you. This project asks you to build the same system a second time, with the framework, on the same documents and the same questions, and then to say what the framework gave you and what it took away.

Part 1 is practice: work through the two lecture notebooks in this module, `langchain_basics` and `langchain_rag`, then finish five short pieces of code in the starter notebook. The rest is the project proper. You reuse the documents and questions you created in the RAG module, rebuild the question and answer bot with LangChain, measure it the same way, and compare the two versions in a report and a video.

Everything graded runs offline on a small local embedding model, with no API key and no GPU. If you have a key or a local model, the last stage of the chain will use it.

2. Using AI assistants

You may use ChatGPT, Claude, Copilot, or any other assistant to write code, fix errors, and organize results. What is assessed is whether you understand what you built. In the video you will walk through your own notebook and explain what each stage does and which of your earlier functions it replaced. Anything you cannot explain will not earn credit, however well it runs.

3. Part 1: practice with the LangChain pipeline

10 points

Open `MiniProject_FrameworkRAG_Starter.ipynb`. Parts 1 to 5 contain five short pieces of code, each marked with a `TODO` banner, and each replacing something you wrote by hand in the RAG module. Work from the two lecture notebooks in this module: `langchain_basics` covers prompt templates, models, output parsers, and the pipe, and `langchain_rag` covers the loader, splitter, vector store, and retriever.

	You write	Replaces
TODO 1	load the file and split it	your chunking loop
TODO 2	build the vector store	your embedding and add calls
TODO 3	make a retriever with <code>k</code> and a score threshold	your retrieve function
TODO 4	write the prompt template	your prompt assembly
TODO 5	wire the chain with the pipe operator	your ask function

The right hand column is the point of the exercise. Keep notes as you go, because the report asks you to name what each stage replaced.

One difference to watch. In the RAG module you kept hits below a maximum distance. LangChain keeps hits above a `score_threshold`, which is a relevance score where higher means more similar. Same idea, opposite direction, and getting it backwards is the most common mistake in TODO 3.

4. Part 2: rebuild your own RAG with LangChain

30 points

From Part 8 of the notebook onward, no code is provided.

Copy the `my_kb/` folder and the `goldset.json` you created in the RAG module into this project folder. Do not change the documents or the questions. The comparison depends on both bots answering exactly the same eight questions over exactly the same text.

Then recreate the question and answer bot with LangChain: load and split your documents, build a vector store, wrap it in a retriever, write a prompt template, and wire the chain. Print a transcript of your bot answering several of your questions, including at least one it should refuse.

5. Part 3: measure it the same way

15 points

Score the same two things you scored in the RAG module: how many answerable questions were found, and how many unanswerable ones were refused. Vary one setting at a time, such as the score threshold, the number of chunks retrieved, or the chunk size, and record the results in a table saved as `results_langchain.csv`. Keep at least one configuration that matches the settings you used in the RAG module, so you have a like for like row in your comparison.

6. What to hand in

Submit this

one PDF report. On its first page, put the link to your project folder on GitHub and the link to your video. Everything else lives in the repository.

6.1 Your repository (5 points)

Use the private repository you created for the RAG module, with your instructor and the class TA as collaborators, and add a new folder for this project alongside the earlier one:

```
your-repo/
  rag_project/           from the RAG module
  langchain_project/
    README.md            what the project is, in a short paragraph
    langchain_notebook.ipynb your completed notebook, with all outputs saved
    my_kb/               the same documents as the RAG module
    goldset.json         the same eight questions
    results_langchain.csv your results
```

Run the notebook top to bottom and save it with its outputs showing before you commit.

6.2 Report (25 points)

A PDF. Begin with the concept check, five short answers of two or three sentences each, in your own words:

- What problem does LangChain solve, compared with calling a model API directly?
- What is a prompt template, and why use placeholders instead of a hard coded prompt?
- What does a chain of prompt, model, and output parser do, stage by stage?
- What is a retriever, and how does it plug into a chain?
- When would you use a branch rather than running two steps in parallel? Give a security example of each.

Then the comparison, which is most of the credit:

1. What you built, briefly, and confirmation that the documents and questions are unchanged.
2. What LangChain replaced. Go through the five stages and name the code from your RAG module notebook that each one made unnecessary.
3. Where the framework helped and where it hid something. Be specific about what you can no longer see, such as how chunk boundaries were chosen or how a score was computed, and say when that would matter.
4. The numbers. Your results table next to your RAG module results on the same questions. Where the two differ, explain why, remembering that a distance and a relevance score run in opposite directions.
5. Which version you would build next time, and for what kind of work.
6. One question this version got wrong. Look at what it retrieved instead and explain the mistake.

6.3 Video (15 points)

Five to ten minutes of screen recording with your narration. Walk through your notebook while it runs, explain the chain stage by stage in your own words, show your system answering one question and refusing another, and talk through your comparison. Live screens, not slides of screenshots.

7. Grading

Part	Points
Part 1, practice pipeline: five TODOs complete and the notebook running	10
Part 2, your own documents and questions rebuilt with LangChain, end to end	30
Part 3, the same measurements, <code>results_langchain.csv</code> , and a like for like row	15
Repository: private, instructor and TA added, separate folder, outputs saved	5
Report, including the comparison of the two versions	25
Video	15
Total	100

Keep API keys in `.env` or Colab Secrets, never in the notebook or the repository. Build this version with LangChain: a hand written pipeline is the RAG module, not this one.